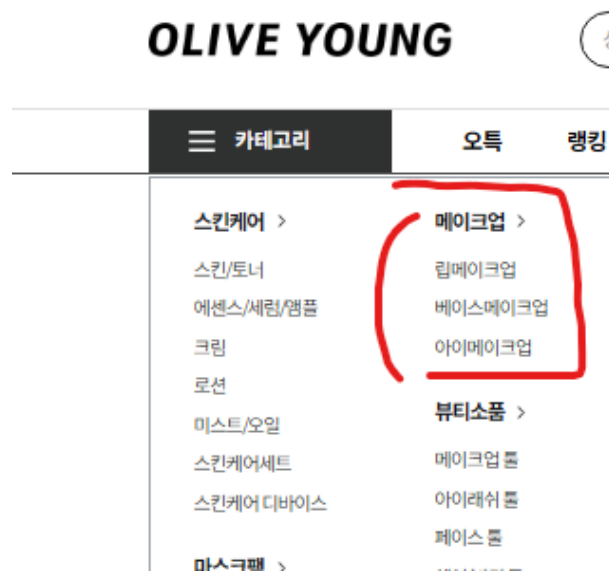


크롤링 코드 구성

📅 날짜	May 28, 2026 → May 29, 2026
⚙️ 상태	진행 중
☰ 태그	
🕒 생성 일시	May 28, 2026 11:10 PM

올리브영 > 카테고리 > 립메이크업 / 베이스메이크업 / 아이메이크업 으로 되어있음



[브랜드 갯수 정리]

▼ 립메이크업

- 립틴트(93)
- 립스틱(53)
- 립라이너(22)
- 립케어(169)
- 컬러립밤(88)
- 립글로스 (52)

▼ 베이스 메이크업

- 쿠션 (85)
- 파운데이션 (52)
- 블러셔 (82)
- 파우더 / 팩트 (51)
- 컨실러 (41)
- 프라이어머 / 베이스 (72)
- 셰이딩 (26)
- 하이라이터 (42)
- 메이크업 픽서 (19)
- BB / CC (42)

▼ 아이메이크업

아이라이너 (50)

마스카라 (53)

아이브로우 (62)

아이세도우 (75)

아아래쉬 케어 (54)

아아 픽서 (29)

크롤링 구성

크롤링은 두단계로 나눌 예정



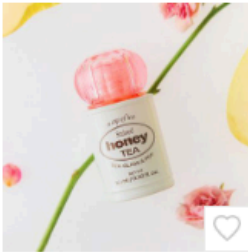
- **목록 수집 봇 (URL 헌터):** * 사용자가 '틴트'라고 입력하면 검색 결과 페이지를 1 페이지부터 끝까지 돌면서 상품 상세페이지 URL들만 짝 긁어옵니다.
 - 긁어온 URL들을 DB의 임시 테이블이나 배열(`List`)에 저장합니다.
- **상세 수집 봇 (데이터 추출기 - 앞서 만든 코드):**
 - 저장된 URL 리스트를 `for` 문으로 하나씩 꺼냅니다.
 - `get_oliveyoung_product_data(url)` 함수에 넣고 실행합니다.
 - 추출된 상품 정보와 옵션(스와치 이미지) JSON을 최종 DB 테이블 (`Product`, `ProductOption`)에 `INSERT` 합니다.

검색 결과 페이지에 상품이 100개가 있다고 해서 반복문으로 100개를 쉬지 않고 크롤링하면 바로 차단당합니다. 반드시 1건의 상세 페이지를 크롤링하고 나면 `time.sleep(5)` 이상을 주어 숨을 고르고 다음 URL로 넘어가도록 코드를 설계하셔야 합니다.

1단계: 카테고리 > 메이크업 > 립메이크업 > 립틴트(여기까지) 내려온 다음에 크롤링을 진행할 예정
제품별 링크들을 다 수집함 → 리스트화

인기순 | 신상품순 | 판매순 | 낮은가격순 | 할인율순

VIEW 24 36 48



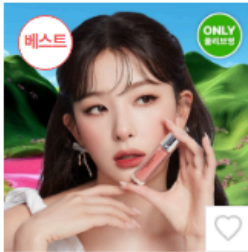
얼터너티브스테레오
얼터너티브스테레오 립 포션 허니티 스테인 5ml
19,000원-
오늘드림



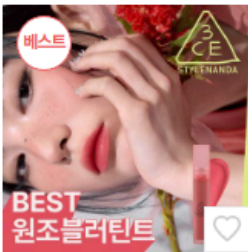
콜맨
[NEW듀우코이] 콜맨더 지시 래스팅 틴트 단품/기획 (+미니 틴트 증정)
13,000원- 9,900원-
세일 중점 오늘드림



얼터너티브스테레오
[NEW초여름 컬러/6월 올영픽] 얼터너티브스테레오 립 포션 카라멜
19,000원- 15,200원-
세일 쿠폰 오늘드림



오아드
[올영단독컬러/NEW/슬기PICK] 오아드 슬레인 파우더믹싱 워터틴트 8
23,000원- 16,900원-
세일 오늘드림



3CE
[로즈코이/컬색생기] 3CE 블러 워터 틴트 4.6g 20종 단품/기획
19,000원- 15,200원-
세일 오늘드림



뽀아
[5월 올영픽/포켓몬 에디션] 뽀아 오버 클레이즈 18종 (기획/단품)
16,000원- 11,800원-
세일 쿠폰 오늘드림



컬러그램
[5월 올영픽/포켓몬 에디션/파우치 증정] 컬러그램 누디 블러 틴트
10,000원- 8,600원-
세일 쿠폰 오늘드림



페리페라
[NEW-컬러] 페리페라 무드 글로이 틴트 27 Colors (단품/기획)
13,000원-
오늘드림

2단계: 링크리스트에서 1개를 들어감 거기서 세부 옵션까지 들어간 내용 → 리스트

제품 구성 상 옵션에서 이미지 링크로 발색을 보여주고,
또 color map 혹은 shade chart 라는 도표를 통해 (쿨-웜, 라이트-딥)을
통해
색이 어떻게 퍼져 있는지 확인할 수 있게 해준다

1. 크롤링과 데이터 가공(AI) 파이프라인의 분리

가장 중요한 핵심은 '웹 크롤러가 색상을 판단하게 하지 않는 것'입니다.

크롤러의 역할은 '원시 데이터(텍스트 + 이미지 URL)'를 수집하여

DB에 적재하는 것까지로 제한하고,

이후 AI 모델이 수집된 이미지를 분석해 색상 데이터를 추출하도록

파이프라인을 분리해야 합니다.

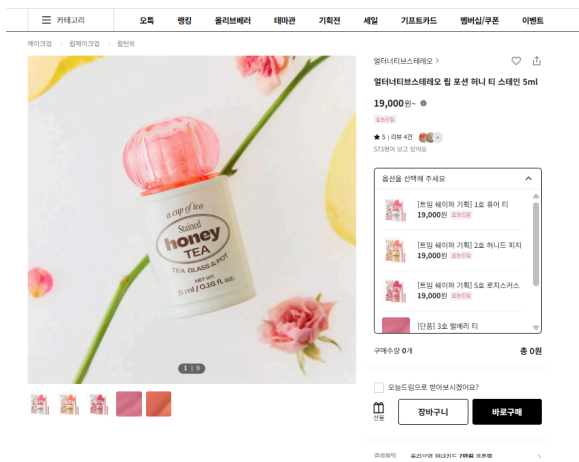
Phase 1 (Data Extraction):

크롤러가 올리브영에서 상품 기본 정보와 '옵션별 이미지(발색/쌈네일)' URL을 수집하여 DB에 저장 (match_score, colormetric_id, personalcolor_id는 일단 NULL 처리).

Phase 2 (AI Processing):

백엔드에서 productoption_image의 URL을 다운로드하여 ResNet, SAM, VLM 등의 모델에 통과시킴.

모델이 추출한 컬러값을 바탕으로 colormetric_id와 personalcolor_id를 업데이트.



예상되는 데이터 형식

```
{  
  "product_id": "A000000123456",  
}
```

```

"brand": "롬앤",
"name": "쥬시 래스팅 틴트",
"base_url": "https://www.oliveyoung.co.kr/...",
"category_id": "C01",
"options": [
    {
        "productoption_id": "OPT_001",
        "productoption_name": "06 피그피그",
        "productoption_image": "https://image.oliveyoung.co.kr/.../figfig_thumb.jpg"
    },
    {
        "productoption_id": "OPT_002",
        "productoption_name": "07 쥬쥬브",
        "productoption_image": "https://image.oliveyoung.co.kr/.../jujube_thumb.jpg"
    }
]
}

```

크롤링 1단계

```

import time
from selenium import webdriver
from selenium.webdriver.chrome.service import Service
from selenium.webdriver.chrome.options import Options
from selenium.webdriver.common.by import By
from webdriver_manager.chrome import ChromeDriverManager

def collect_product_urls(keyword, max_pages=1):
    # 1. 봇 차단 우회 및 브라우저 환경 설정
    chrome_options = Options()
    chrome_options.add_argument("user-agent=Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36")

```

```

chrome_options.add_argument("--disable-blink-features=AutomationControlled")

service = Service(ChromeDriverManager().install())
driver = webdriver.Chrome(service=service, options=chrome_options)

collected_urls = []

try:
    # 지정한 페이지 수(max_pages)만큼 반복하며 수집
    for page in range(1, max_pages + 1):
        print(f"[{keyword}] 검색 결과 {page}페이지 수집
중...")

        # 올리브영 검색 URL (실제 페이지네이션 파라미터가 있다면
        URL에 추가 필요)
        search_url = f"https://www.oliveyoung.co.kr/store/search/getSearchMain.do?query={keyword}"

        # 2. 페이지 이동 및 대기 (대상 서버의 인프라 보호 및 안정성 확보)
        driver.get(search_url)
        time.sleep(5) # robots.txt 규약 준수 및 DOM 렌더링 대기

        # 3. 상품 상세 페이지로 이어지는 a 태그 추출
        # (주의: 실제 웹페이지 F12에서 썸네일 이미지나 상품명을 감싸는 a 태그의 클래스명 확인 후 변경하세요)
        items = driver.find_elements(By.CSS_SELECTOR, "a.prd_thumb")

        if not items:
            print("더 이상 수집할 상품이 없습니다.")
            break

```

```

        # 4. URL 추출 및 필터링
        for item in items:
            link = item.get_attribute('href')
            # 가짜 링크(자바스크립트 동작)가 아닌 실제 URL만 유효성 검사 후 저장
            if link and "javascript:" not in link:
                collected_urls.append(link)

        print(f"{page}페이지에서 {len(items)}개의 URL 확보 완료.")

    except Exception as e:
        print(f"URL 수집 중 시스템 에러 발생: {e}")
    finally:
        driver.quit()
        print("1단계 URL 수집 세션 종료 및 자원 반환.")

# 5. 중복 수집된 URL 제거 후 반환
return list(set(collected_urls))

# =====
# 실행 테스트 구간
# =====
if __name__ == "__main__":
    # '틴트' 검색 결과를 1페이지만 수집하도록 테스트
    target_keyword = "틴트"
    urls = collect_product_urls(target_keyword, max_pages=1)

    print(f"\n[최종 확보된 타겟 URL 리스트 - 총 {len(urls)}건]")
    for url in urls:
        print(url)

```

[최종 1차 크롤링 코드]

```
import json
```



```

import time
import os
from selenium import webdriver
from selenium.webdriver.chrome.service import Service
from selenium.webdriver.chrome.options import Options
from selenium.webdriver.common.by import By
from webdriver_manager.chrome import ChromeDriverManager

def collect_urls_by_category(category_id, max_pages=10):
    # 1. 안정성을 위한 브라우저 환경 설정
    chrome_options = Options()
    chrome_options.add_argument("user-agent=Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36")
    chrome_options.add_argument("--disable-blink-features=AutomationControlled")

    service = Service(ChromeDriverManager().install())
    driver = webdriver.Chrome(service=service, options=chrome_options)

    collected_urls = []

    try:
        # 1페이지부터 max_pages까지 순회
        for page in range(1, max_pages + 1):
            print(f"[카테고리 {category_id}] {page}페이지 수집
중...")

            # 2. pageIdx 파라미터를 동적으로 변경하여 URL 생성
            # 불필요한 트래킹 파라미터는 제거하고 핵심 데이터만 남긴
            # 깔끔한 URL
            target_url = (
                f"https://www.oliveyoung.co.kr/store/display/getMCategoryList.do?"
                f"dispCatNo={category_id}&prdSort=01&pageIdx="

```

```

{page}&rowsPerPage=24"
    )

    # 페이지 접근 및 대기 (5초 딜레이)
    driver.get(target_url)
    time.sleep(5)

    # 3. 상품 상세 페이지 링크 추출
    # 올리브영 카테고리 페이지의 상품 링크 a 태그 선택
    items = driver.find_elements(By.CSS_SELECTOR, "a.
prd_thumb")

    # 만약 해당 페이지에 수집할 요소가 하나도 없다면 마지막
    페이지를 넘긴 것이므로 탈출
    if len(items) == 0:
        print(f"{page}페이지에 더 이상 상품이 없습니다. 수
집을 조기 종료합니다.")
        break

    # 4. 링크 저장 및 검증
    valid_count = 0
    for item in items:
        link = item.get_attribute('href')
        if link and "javascript:" not in link:
            collected_urls.append(link)
            valid_count += 1

    print(f" -> {page}페이지에서 {valid_count}개의 유효
한 URL 확보 완료.")

except Exception as e:
    print(f"URL 수집 중 시스템 에러 발생: {e}")
finally:
    driver.quit()
    print("\n수집 세션 종료 및 브라우저 자원 반환 완료.")

```

```

# 중복 제거 후 반환
return list(set(collected_urls))

# =====
# 실행 테스트 구간
# =====
if __name__ == "__main__":
    # 타겟 카테고리 ID (올려주신 링크의 dispCatNo 값: 립틴트)
    TARGET_CATEGORIES = {
        # "쿠션": "1000001000200010009",
        # "파운데이션": "1000001000200010002",
        # "블러셔": "1000001000200010006",
        # "파우더_팩트": "1000001000200010004",
        # "컨실러": "1000001000200010005",
        # "쉐이딩": "1000001000200010007",
        # "하이라이터": "1000001000200010008",
        "아이라이너": "1000001000200070002",
        "마스카라": "1000001000200070001",
        "아이브로우": "1000001000200070004",
        "아이섀도우": "1000001000200070003"
    }

    # 바탕화면이나 프로젝트 폴더에 파일들이 섞이지 않도록 전용 폴더를
    # 하나 만듭니다.
    SAVE_DIR = "./url_datasets"
    if not os.path.exists(SAVE_DIR):
        os.makedirs(SAVE_DIR)

    print("🚀 다중 카테고리 개별 저장 파이프라인을 시작합니다...\n")

    for category_name, category_id in TARGET_CATEGORIES.items():
        print(f"===== [{category_name}] 카테고리 수집 시작
        =====")

```

```

# 1. URL 수집
urls = collect_urls_by_category(category_id=category_
id, max_pages=8)

# 2. 카테고리 이름으로 개별 파일명 생성 (예: url_datasets/
올리브영_립틴트_urls.json)
file_name = f"{SAVE_DIR}/올리브영_{category_name}_url
s.json"

# 3. 수집 즉시 개별 파일로 저장
# 큰 바구니(딕셔너리)에 모이지 않고, 리스트 자체를 바로 파일
에 씁니다.
with open(file_name, "w", encoding="utf-8") as f:
    json.dump(urls, f, ensure_ascii=False, indent=4)

print(f"✅ [{category_name}] 총 {len(urls)}건 -> '{fil
e_name}'에 독립 저장 완료.\n")

# 서버 차단 방지 휴식
time.sleep(10)

print("🎉 모든 카테고리의 개별 파일 생성이 완료되었습니다!")

```



⚙️ 코드가 동작하는 핵심 원리

이 코드는 대규모 트래픽을 유발하지 않으면서 목적을 달성하는 '정찰병(Scout)' 역할을 합니다.

1. 위장 및 진입 (환경 설정):

서버 방화벽(WAF)이나 봇 탐지 시스템이 비정상적인 접근으로 간주하지 않도록, `User-Agent` 를 일반 윈도우 크롬 브라우저로 변조하여 올리브영 검색 페이지에 진입합니다.

2. 보수적 지연 대기 (`time.sleep`): `driver.get()` 으로 페이지를 요청한 직후 무조건 5초를 대기합니다. 이는 `robots.txt` 의 `Crawl-delay: 5` 권고를 준수함과 동시에, 브라우저가 화면의 상품 이미지와 HTML 요소(DOM)를 모두 렌더링할 때까지 안전하게 기다리는 역할을 합니다.

3. 요소 탐색 및 타겟팅 (DOM 파싱): `driver.find_elements` 를 통해 화면에 깔린 수십 개의 상품 리스트 중 상품 클릭 시 상세 페이지로 넘어가는 하이퍼링크 (`<a href="...">`) 요소만 핀셋처럼 집어냅니다.

4. 검증 및 적재 (필터링):

추출한 링크 중에 페이지 내부에서만 작동하는 자바스크립트 찌꺼기 코드 (`javascript:void(0)` 등)가 섞여 있을 수 있으므로 이를 걸러냅니다. 순수 URL 만 추출한 뒤, `set()` 함수를 사용해 혹시라도 두 번 수집된 중복 URL을 말끔히 제거하고 최종 배열로 반환합니다.

크롤링 2단계

```

import time
import json
from selenium import webdriver
from selenium.webdriver.chrome.service import Service
from selenium.webdriver.chrome.options import Options
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from webdriver_manager.chrome import ChromeDriverManager

```

```

def extract_product_details(url_list):
    # 1. 봇 탐지 우회 및 드라이버 설정
    chrome_options = Options()
    # 대량 수집 시에는 화면을 띄우지 않는 headless 모드를 켜는 것이
    메모리 관리에 좋습니다.
    # 단, 초기 디버깅 시에는 주석 처리하여 눈으로 확인하세요.
    # chrome_options.add_argument("--headless")
    chrome_options.add_argument("user-agent=Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36")
    chrome_options.add_argument("--disable-blink-features=AutomationControlled")

    service = Service(ChromeDriverManager().install())
    driver = webdriver.Chrome(service=service, options=chrome_options)

    # 최종 수집된 데이터를 담을 리스트
    final_dataset = []

    # 2. 전달받은 URL 리스트 순회 (Worker Loop)
    for index, product_url in enumerate(url_list, 1):
        print(f"[{index}/{len(url_list)}] 데이터 추출 중: {product_url}")

        product_data = {
            "product_id": f"P{str(index).zfill(5)}", # 임시 ID
            # (실제 DB의 Auto Increment 또는 URL의 상품코드 파싱 권장)
            "base_url": product_url,
            "options": []
        }

        try:
            # 3. 보수적인 서버 접근 (Crawl-delay 규칙 엄수)
            driver.get(product_url)

```

```

        time.sleep(5) # 각 상세 페이지 접근마다 무조건 5초 대기 (IP 차단 방지)

        wait = WebDriverWait(driver, 10)

        # 4. 부모 테이블(Product) 정보 추출
        try:
            brand_elem = wait.until(EC.presence_of_element_located((By.CSS_SELECTOR, "p.prn_brand")))
            product_data['brand'] = brand_elem.text
        except:
            product_data['brand'] = "브랜드 미상"

        try:
            name_elem = wait.until(EC.presence_of_element_located((By.CSS_SELECTOR, "p.prn_name")))
            product_data['name'] = name_elem.text
        except:
            product_data['name'] = "상품명 파싱 실패"

        # 5. 자식 테이블(ProductOption) 정보 추출
        try:
            # 옵션 창 열기 (드롭다운 형태일 경우)
            option_dropdown = wait.until(EC.element_to_be_clickable((By.CSS_SELECTOR, "div.option_title")))
            option_dropdown.click()
            time.sleep(2) # 렌더링 애니메이션 대기
        except:
            # 옵션 창이 없거나 단일 상품인 경우 에러를 무시하고 넘어감 (결함 격리)
            pass

        # 펼쳐진 옵션 리스트 추출
        option_elements = driver.find_elements(By.CSS_SELECTOR, "ul.option_list > li")

```

```

        for opt_idx, option in enumerate(option_elements,
1):
            try:
                opt_name = option.find_element(By.CSS_SELECTOR, "span.option_value").text

                # 스위치 이미지 추출 (썸네일이 <img> 태그 안에
있다고 가정)
                img_element = option.find_element(By.CSS_SELECTOR, "img")
                img_url = img_element.get_attribute('src')

                product_data['options'].append({
                    "productoption_id": f"{product_data
['product_id']}_OPT{str(opt_idx).zfill(3)}",
                    "productoption_name": opt_name,
                    "productoption_image": img_url,
                    # 이후 AI 파이프라인에서 채워질 영역 (기본
값 null)
                    "match_score": None,
                    "personalcolor_id": None,
                    "colormetric_id": None
                })
            except Exception as inner_e:
                # 특정 옵션 하나가 파싱에 실패해도 전체 반복문은
계속 돌아가도록 처리
                continue

            final_dataset.append(product_data)
            print(f" -> 성공: {len(product_data['options'])}개
의 옵션 수집 완료.")

        except Exception as e:
            # 특정 URL에서 치명적 에러가 나도 다음 URL로 넘어가도록
처리

```



```

        print(f" -> 실패: 해당 URL 크롤링 중 에러 발생
({e}))")
        continue

    driver.quit()
    print("\n✅ 2단계 상세 추출 작업이 모두 완료되었습니다.")

    return final_dataset

# =====
# 실행 테스트 구간 (1단계와 2단계의 연동 시뮬레이션)
# =====
if __name__ == "__main__":
    # 1단계에서 수집했다고 가정하는 샘플 URL 리스트 (실제로는 1단계
    # 함수의 리턴값을 받습니다)
    sample_urls = [
        "https://www.oliveyoung.co.kr/store/goods/getGoodsDetail.do?goodsNo=A000000180567",
        "https://www.oliveyoung.co.kr/store/goods/getGoodsDetail.do?goodsNo=A000000192345"
    ]

    # 2단계 작업 실행
    result_data = extract_product_details(sample_urls)

    # 결과를 JSON 파일로 안전하게 저장 (DB 적재 전 중간 기착지 역할)
    with open('oliveyoung_cosmetics_data.json', 'w', encoding='utf-8') as f:
        json.dump(result_data, f, ensure_ascii=False, indent=2)

    print("결과가 oliveyoung_cosmetics_data.json 파일로 저장되었습니다.")

```



⚙️ 2단계 코드의 핵심 동작 원리 (시스템 안정성 확보)

1. 상태 격리 (Try-Except의 계층화):

코드를 보시면 `try-except` 블록이 3중으로 겹쳐 있습니다. 브랜드명을 못 찾았다고 해서 상품명 추출을 포기하지 않으며, 특정 옵션 하나의 이미지가 깨졌다고 해서 다른 정상적인 옵션들의 파싱을 멈추지 않습니다. 최악의 경우 해당 상품 전체에서 에러가 나도 다음 URL 검색으로 조용히 넘어갑니다. 시스템이 절대 중간에 죽지 않도록 설계되었습니다.

2. AI 데이터 전처리 호환성:

반환되는 JSON 구조를 확인해 보시면 부모와 자식 테이블의 정규화된 형태를 그대로 띄고 있습니다. 이 구조는 추후 데이터베이스에 적재하기도 편할 뿐만 아니라, 수집된 스와치 이미지 URL들을 다운로드하여 비전 언어 모델(VLM)이나 Grad-CAM 등을 활용해 퍼스널 컬러와 RGB 메트릭을 추출하는 다음 AI 파이프라인으로 넘기기에 최적화된 상태입니다.

3. 파일 기반 버퍼링:

실행 테스트 구간을 보면 추출이 끝난 데이터를 바로 DB에 꽂지 않고 일단 `json` 파일로 저장합니다. 크롤링 도중 DB 커넥션이 끊어지는 등의 인프라 이슈를 방지하고, 원시 데이터(Raw Data)를 안전하게 백업해두는 엔지니어링 관점의 안전장치입니다. 이후 별도의 스크립트를 통해 이 JSON 파일을 읽어 DB에 `INSERT` 하는 것이 훨씬 안정적입니다.